

Министерство образования и науки Российской Федерации
Московский физико-технический институт (государственный университет)

Физтех-школа радиотехники и компьютерных технологий
Кафедра микропроцессорных технологий в интеллектуальных системах управления

Выпускная квалификационная работа магистра

Изменение трасс исполнения для модификации записанного образа памяти

Автор:

Студент 407 группы
Матренин Василий Николаевич

Научный руководитель:

Петушков Игорь Вячеславович

Научный консультант:

Матвеев Сергей Владимирович



Москва 2026

Трасса исполнения представляет собой набор бинарных или текстовых файлов, содержащих упорядоченные записи о событиях, происходивших в ходе трассировочного запуска целевого приложения. Трассы исполнения широко применяются в индустрии для решения различных задач, таких как: исследование поведения приложений, отладка, сбор профилей исполнения для компиляторов.

Возникает необходимость анализа трасс исполнения, при настройках адресации, отличных от настроек исходной платформы, - платформы, на которой трассы были записаны. Подобный подход может быть использован для оценки производительности еще не выпущенных платформ при различных конфигурациях адресации.

В рамках данной работы описано решение, модифицирующее трассы исполнения для изменения образов памяти записанных сценариев исполнения. Решение позволяет перемещать области исходного образа памяти для удовлетворения ограничений, обусловленных новыми настройками адресации.

Для генерации необходимых модификаций трасс используется граф потока данных. Реализованы алгоритмы для: генерации карты перемещений областей памяти, сбора графа потока данных с трассы исполнения, поиска локаций адресов и смещений на графе, обработки краевых случаев с помощью симуляционного прохода по модифицированной трассе. Описанный подход позволяет существенно снизить влияние модификаций на репрезентативность модифицированных трасс и может быть использован для других задач, связанных с модификацией адресов в трассах исполнения.

Предложенное решение успешно применяется для анализа записанных сценариев исполнения с увеличенными размерами страниц адресации и расширенными областями, зарезервированными под данные ядра операционной системы. Произведена оценка влияния модификаций трасс на их репрезентативность. Предложены возможности для дальнейшего улучшения решения и новые направления для его применения.

+ССЫЛКИ

Содержание

Аббревиатуры	5
Математические обозначения	5
Словарь терминов	5
1 Введение	7
1.1 Трассы исполнения	7
1.2 Виртуальное адресное пространство	8
1.3 Цель работы	10
2 Обзор существующих решений	11
2.1 Позиционно-независимый код	11
2.2 Компоновщики	11
2.3 Компиляторы	12
2.4 Сборщики мусора ВМ	13
2.5 Динамическое восстановление вырезанных таблиц перемещений	13
3 Решение задачи	15
3.1 Архитектура решения	15
3.2 Генерация карты перемещений	16
3.3 Сбор графа потока данных	17
3.4 Генерация модификаций трассы	21
3.4.1 Поиск адресов и смещений	23
3.4.2 Симуляционный проход по модифицированной трассе	25
3.5 Запись модифицированной трассы	26
3.6 Симуляция по трассе	26
3.7 Тестирование	28
3.8 Оценка решения	32
3.8.1 Пространственная и временная сложность	32
3.8.2 Оценка репрезентативности трасс	33
3.8.3 Профилировка решения по памяти	33
3.8.4 Тестовые запуски на целевых трассах	34
3.9 Возможные направления дальнейшего развития решения	35

3.9.1	Поддержка краевых случаев	35
3.9.2	Оптимизация по памяти	36
3.9.3	Поддержка векторных регистров	37
3.9.4	Модификация криптографических подписей адресов	37
4	Заключение	38

Аббревиатуры

ASLR Address Space Layout Randomization, рандомизация адресного пространства. 11

GOT Global Offset Table, таблица глобальных смещений. 11

PIC Position Independent Code, позиционно независимый код. 11

PLT Procedure Linkage Table, таблица связывания процедур. 11

ВМ Языковая Виртуальная Машина. 13

ГПД Граф Потока Данных. Вершины графа моделируют исполнения инструкций (иногда называемые просто инструкциями), ребра - зависимости между инструкциями по значениям. С каждым ребром ассоциируются значение и локация, по которой данное значение располагается (также называемые значением и локацией ребра).. 15

ОС Операционная Система. 8

ПО Программное обеспечение. 28

Математические обозначения

$G = (V, E)$ ГПД, где: **V** - множество вершин графа, **E** - множество ребер графа. 17

2^E Множество всех подмножеств ребер ГПД. 17

Словарь терминов

Адресное ребро Ребро ГПД, значение которого используется в качестве адреса. 23

Исходное адресное ребро Ребро, на котором заканчивается построение обратного пути из адресного ребра на ГПД в алгоритме поиска адресов. 23

Модуль Библиотека или приложение. 11

Модульное тестирование (module testing) Тестирование группы связанных компонентов, образующих законченный модуль системы. Проверяет корректность взаимодействия внутри модуля и его интерфейсов. 28

Псевдо-инструкция Вершина ГПД, не отвечающая реальной инструкции. Используется для отображения различных зависимостей на графе. 17

Сквозное тестирование (end to end testing) Тестирование полного бизнес-сценария в условиях, максимально приближенных к реальным. Проверяет работу всех компонентов системы вместе, включая внешние зависимости. 28

Трасса исполнения Набор упорядоченных записей о событиях, происходивших во время трассировочного запуска приложения. 7

Юнит тестирование (unit testing) Тестирование минимального фрагмента кода (функции или метода) в изоляции от других компонентов. Поведение внешних зависимостей имитируется с помощью объектов-подражателей (mock objects). 28

1 Введение

1.1 Трассы исполнения

Трасса исполнения представляет собой набор упорядоченных записей о событиях: исполнениях инструкций, операциях с памятью, системных вызовах, сигналах и т. д., происшедших во время трассировочного запуска приложения.

Трассы исполнения применяются в индустрии для различных задач, таких как: микро-архитектурный и макро анализ приложений [6] [3] [1], сбор профилей исполнения [7], отладка [5].

Трасса может содержать записи для нескольких потоков исполнения, однако для простоты в дальнейшем будем рассматривать только трассы исполнения с одним потоком. Описанное в данной работе решение нетрудно обобщается на трассы с произвольным числом потоков.

Рассматриваемые трассы исполнения содержат следующие основные виды записей:

- Начальное регистровое состояние

Содержит значения регистров общего назначения, векторных регистров и условных флагов в момент начала трассировки. Также такие записи используются для отображения состояния машины при переключениях контекста исполнения (обработка сигналов, изменение уровня привилегий).

- Исполнение инструкции

Содержит информацию об исполненной инструкции: адрес, кодировку и номер инструкции в трассе. С данной записью могут ассоциироваться другие записи, связанные с исполнением инструкции (обращения в память, записи в регистры).

- Обращение в память

Содержит адрес, размер, данные и тип обращения (запись или чтение). Одна инструкция может иметь несколько ассоциированных с ней обращений к памяти. Данные для чтения из памяти соответствуют состоянию памяти **до** исполнения инструкции.

- Запись в регистр общего назначения

Содержит идентификатор регистра и его значение **после** исполнения инструкции. Одна инструкция может иметь несколько ассоциированных с ней записей в регистры. Такие записи используются для отслеживания изменений регистров общего назначения системными (системные вызовы, чтения системных регистров) и другими инструкциями.

Изменения памяти, осуществляемые операционной системой (ОС) или другими потоками исполнения будут отражены в трассе (в записях для рассматриваемого потока) только косвенно: при чтении памяти может быть прочитано новое значение, не соответствующее образу памяти, который наблюдался ранее. На рис. 1 приведен пример трассы исполнения, в том числе иллюстрирующий данную особенность (см. записи 2 и 6).

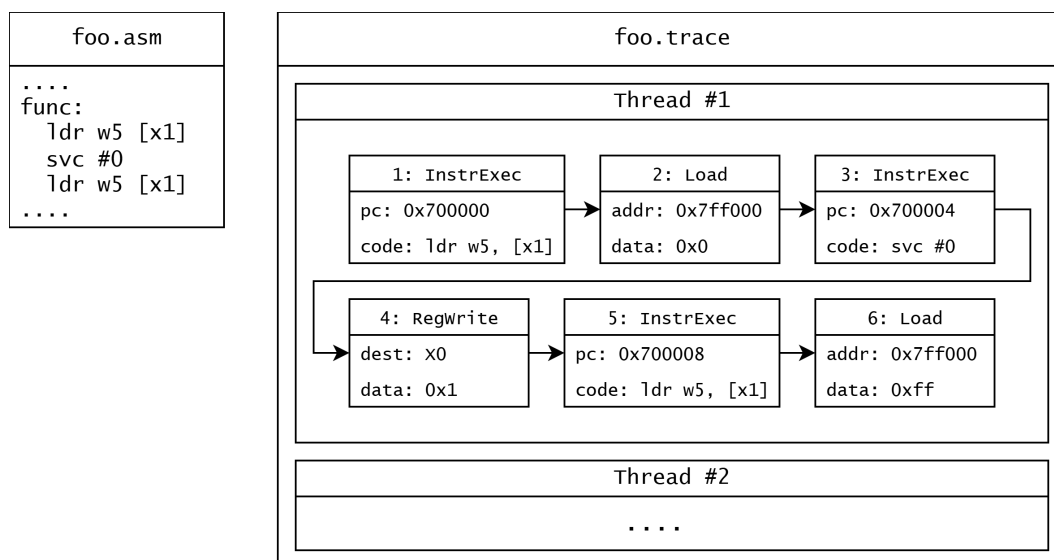


Рис. 1: Пример трассы исполнения.

При анализе трасс исполнения часто используются симуляторы. В таких случаях в состоянии симулятора пробрасываются данные из трассы, для рассматриваемых трасс это: начальное регистровое состояние, коды инструкций, данные для чтений из памяти, новые значения регистров общего назначения. При этом зачастую потоки управления и данных симулятора проверяются на соответствие трассе для детектирования ошибок симуляции или невалидных трасс. Такой подход называется симуляцией по трассе.

Описанные особенности трасс исполнения и их обработки позволяют существенно модифицировать записанные сценарии и получать валидные трассы для дальнейшей обработки: можно изменить или добавить необходимые записи для коррекции потоков управления и данных. На рис. 2 приведен пример модифицированной трассы исполнения. Измененные данные выделены красным цветом.

1.2 Виртуальное адресное пространство

Большинство современных компьютерных систем поддерживает виртуализацию адресного пространства. Данная технология позволяет отображать (транслировать) адреса оперативной памяти, используемые кодом (виртуальные адреса) в адреса аппаратуры (физические адреса). Такой подход позволяет запускать множество приложений изолированно в

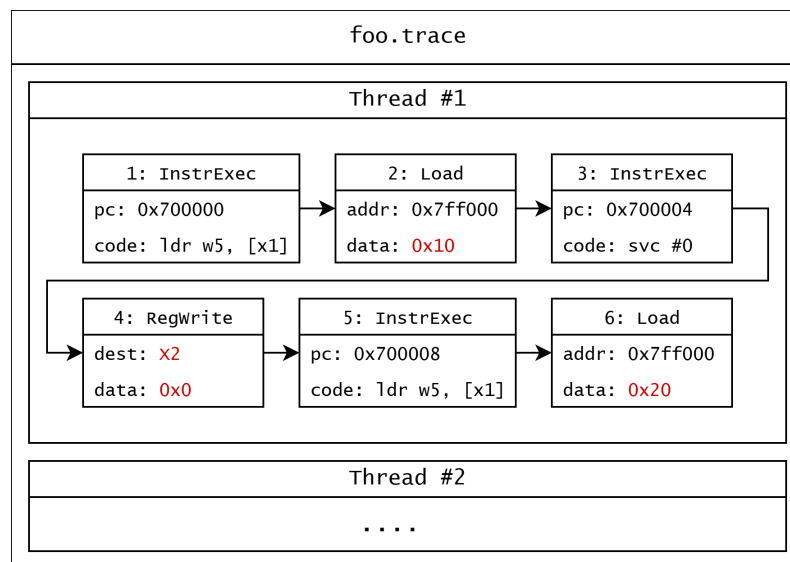


Рис. 2: Пример модифицированной трассы исполнения.

одинаковой среде исполнения. Виртуальные адреса приложений могут пересекаться, однако при обращении к памяти будут использованы разные физические адреса. Создание карты трансляции адресов осуществляет операционная система по запросам приложений и загрузчиков.

При этом трансляция адресов обычно происходит постранично: виртуальное адресное пространство разбивается на непрерывные блоки одинакового размера (виртуальные страницы), последовательные адреса внутри одного блока отображаются в последовательные физические адреса (адреса физической страницы). Размер страницы - очень важный параметр системы, напрямую влияющий на производительность. Уменьшение размера страницы приводит к замедлению трансляции адресов, а увеличение - к большому расходу памяти. Типовые размеры страниц для современных систем: 4096, 16384 и 65536 байт.

К другим настройкам адресации можно отнести расположение запрещенных областей виртуального адресного пространства - областей, использование которых пользовательским кодом запрещено или ограничено. Такие области могут быть заняты данными операционной системы, зарезервированы для будущих расширений, запрещены для детектирования ошибок адресации (null page). Подобные области могут быть доступны из пользовательского кода только на чтение или недоступны вовсе.

Также многие современные системы имеют ограничения на создание отображений с правами на исполнение и запись (WX права) для повышения безопасности.

1.3 Цель работы

Трассы исполнения могут быть использованы для анализа производительности еще не выпущенных платформ. Однако, описанные особенности виртуального адресного пространства существенно усложняют подобный анализ в случае, если настройки адресации разрабатываемой платформы отличаются от настроек адресации платформы, на которой трассы были записаны (исходные настройки адресации). Это мотивирует разработку решения для модификации образов памяти затрассированных сценариев путем изменения трасс исполнения. Заметим, что данная задача сводится к задаче перемещения областей образов памяти трасс в соответствии с новыми ограничениями.

Цель работы: Разработать решение, позволяющее перемещать области образов памяти трассируемых сценариев в соответствии с ограничениями, обусловленными настройками адресации разрабатываемой платформы.

Задачи:

- Разработать модификатор трасс исполнения, перемещающий области образа памяти в соответствии с ограничениями, обусловленными новыми настройками адресации.
- Протестировать работоспособность модификатора трасс на целевых трассах заказчика.
- Оценить влияние модификаций на репрезентативность трасс исполнения.

Работа будет осуществляться в рамках проекта для сбора и анализа трасс исполнения. Будут использованы некоторые компоненты проекта, внешние по отношению к разрабатываемому решению: функциональный симулятор целевой архитектуры, библиотека для чтения и записи трасс исполнения.

2 Обзор существующих решений

В данной главе будут рассмотрены существующие решения в области переноса кода и данных.

2.1 Позиционно-независимый код

Позиционно независимый код (Position Independent Code, PIC) - это код, который может быть размещен по любому адресу. PIC библиотека или приложение (модуль) напрямую не использует фиксированные адреса и для доступа к коду и данным использует относительные смещения или таблицу глобальных смещений (Global Offset Table, GOT) и таблицу связывания процедур (Procedure Linkage Table, PLT). PIC модули могут быть загружены по любым адресам, что позволяет использовать их с технологией рандомизации адресного пространства (Address Space Layout Randomization, ASLR) и существенно упрощает динамическое связывание библиотек.

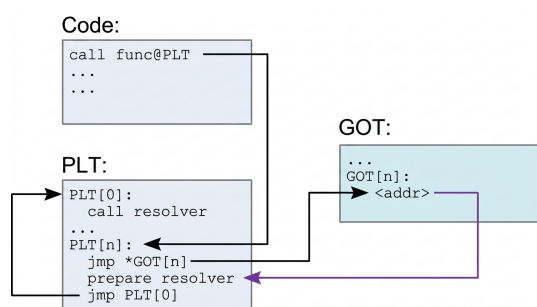


Рис. 3: Схема работы PLT: первый вызов процедуры [2].

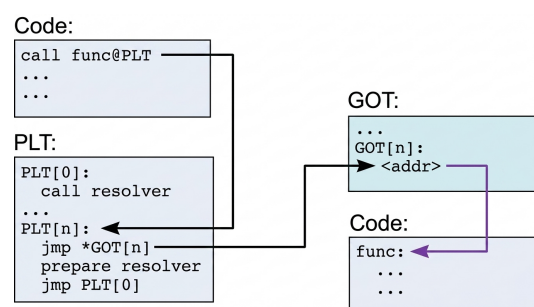


Рис. 4: Схема работы PLT: последующие вызовы [2].

Данная технология используется на целевой платформе, что упрощает модификацию затрассированного образа памяти, так как при переносе PIC сегмента целиком, относительная адресация внутри сегмента будет работать как в оригинальном сценарии (смещения между символами сегмента не изменятся). Однако исходные адреса кода и данных могут остаться на стеке, в куче и GOT. Так же относительная адресация внутри одного модуля перестанет корректно работать в случае переноса сегментов модуля с разными смещениями.

2.2 Компоновщики

В ходе статической компоновки из набора объектных файлов и статических библиотек создается один готовый исполняемый файл или библиотека (статическая или динамическая). Сегменты кода и данных входных файлов объединяются, происходит разрешение символов, с помощью данных из таблиц перемещений производятся замены адресов и смещений в коде и

данных на финальные. Результирующий модуль содержит код и данные входных объектных файлов и статических библиотек, все символы разрешаются при компоновке.

Статическая компоновка позволяет осуществлять интер-модульные оптимизации (post-link optimizations) и уменьшает вычислительные расходы на загрузку приложения так как связывание происходит один раз при компоновке исполняемого файла. При этом данный метод может существенно увеличивать нагрузку на подсистему памяти во время исполнения множества приложений с одинаковыми зависимостями, поскольку код и данные каждой статической библиотеки будут продублированы в каждом приложении.

При динамической компоновке разрешение символов динамических библиотек откладывается до их загрузки (или даже непосредственно до использования). Компоновщик проверяет, что все внешние символы экспортируются какой-то из динамических библиотек, записывает имена динамических библиотек и их символов, создает GOT и PLT. Адреса динамически связываемых символов разрешаются во время их загрузки или использования специально сгенерированным кодом с помощью таблиц перемещений.

Динамическая компоновка позволяет ОС единожды загрузить динамическую библиотеку и предоставить ее код и данные в общее пользование всем зависимым процессам. Применение данного подхода позволяет существенно снижать потребление памяти современных систем. При этом динамическое связывание происходит непосредственно во время исполнения приложения, что приводит к дополнительным вычислительным расходам.

Следует отметить, что в обоих сценариях используются подготовленные компилятором и компоновщиком таблицы перемещений, т.е. локации заменяемых адресов и смещений известны.

2.3 Компиляторы

При однопроходной или двухпроходной компиляции адреса целевых меток кода могут быть неизвестны на момент генерации инструкций перехода на эти метки. Часто для корректного формирования инструкций перехода используется метод отложенного связывания (backpatching). При таком подходе инструкции перехода, для которых неизвестны смещения до целевых меток, генерируются с незаполненными смещениями и локации таких инструкций помещаются в список для дальнейшего исправления. Когда расположение целевой метки становится известным (например после обработки всей единицы трансляции), смещения зависимых инструкций заполняются корректными значениями.

Данный подход позволяет генерировать код без дополнительных полных проходов по промежуточному представлению. В качестве недостатков этого метода можно выделить

затраты памяти на хранение вспомогательной информации и потенциальное усложнение оптимизаций, переставляющих участки кода.

2.4 Сборщики мусора ВМ

Среды выполнения многих языковых виртуальных машин (ВМ) включают сборщики мусора, которые помимо освобождения неиспользуемой памяти могут перемещать используемые данные для решения проблемы фрагментации памяти, улучшения кеш-локальности и других оптимизаций.

Одним из наиболее распространенных семейств алгоритмов сборки мусора является семейство трассирующих сборщиков мусора. Зависимости между объектами, расположенными в памяти виртуальной машины, можно представить в виде направленного графа (граф объектов). Вершины графа - объекты, ребра - ссылки между объектами. Для определения того, какие объекты можно удалять и как обновлять ссылки на еще используемые объекты при их перемещении, осуществляются обходы графа из "корневых" объектов: объектов расположенных на регистрах и стеке ВМ, глобальных объектов, и т. д. Объекты, недостижимые из корневых объектов, можно очищать. А для перемещаемых объектов в ходе такого обхода собираются все ссылки, которые необходимо обновить.

Описанная методология позволяет эффективно освобождать память (в том числе очищать объекты, использующие циклические ссылки) и осуществлять дефрагментацию используемой памяти.

2.5 Динамическое восстановление вырезанных таблиц перемещений

Старые бинарные выпуски библиотек могут являться позиционно зависимыми, что делает системы, которые используют подобные версии библиотек уязвимыми для атак повторного использования кода (code reuse attacks). В [4] представлен метод динамического восстановления таблиц перемещений для позиционно зависимых библиотек, из которых была вырезана данная информация. Технология применяется для частичной рандомизации адресов загрузки позиционно зависимых библиотек, что повышает безопасность уязвимых систем.

При загрузке позиционно зависимой библиотеки код загружается по случайным адресам, а адреса, зарезервированные для кода библиотеки (старые адреса кода), запрещаются для использования через механизмы операционной системы, такие как mmap. При доступе к старым адресам кода производится обработка системного исключения, в ходе которой анализируется паттерн обращения и либо осуществляется замена адреса на перемещенный,

либо обращение помечается как потенциально вредоносное и выводится ошибка.

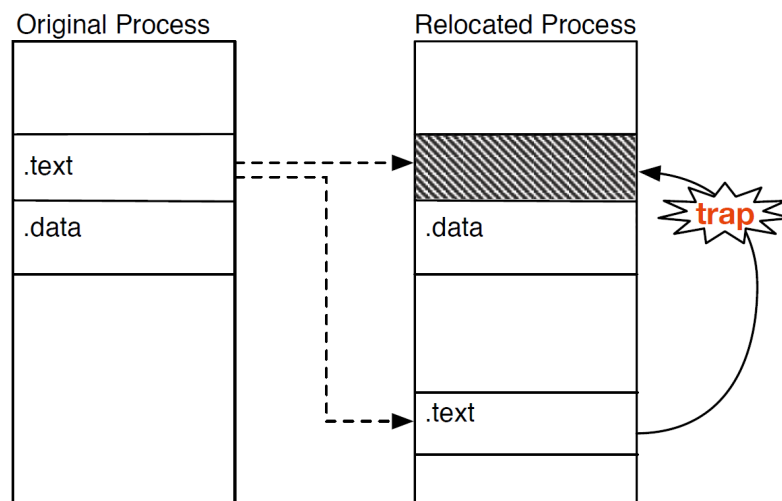


Рис. 5: Схема загрузки позиционно зависимых библиотек с частичной рандомизацией адресов [4]

3 Решение задачи

В данной главе будет рассмотрено решение поставленной задачи. Решение разработано на языке программирования C++.

3.1 Архитектура решения

Общая схема решения представлена на рис. 6.

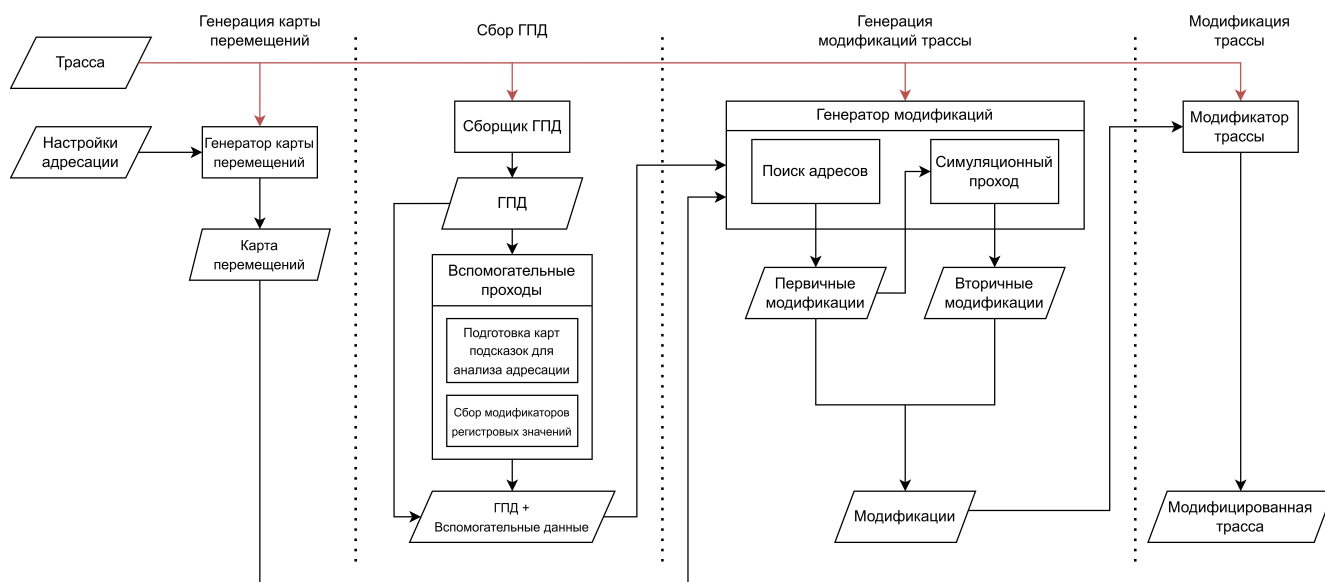


Рис. 6: Общая схема решения.

Обработка трассы происходит в четыре стадии:

1. Генерация карты перемещений

На этой стадии генерируется карта перемещений областей памяти, удовлетворяющая новым ограничениям, обусловленным изменением настроек адресации.

2. Сбор графа потока данных (ГПД)

На данной стадии осуществляется симуляционный проход по трассе, в ходе которого собирается ГПД. Затем осуществляется ряд обходов графа, в ходе которых собирается вспомогательная информация, размечающая его.

3. Генерация модификаций трассы

Генерация происходит в два этапа:

(а) Поиск адресов на ГПД и генерация первичных модификаций

- (b) Симуляционный проход по модифицированной трассе и генерация вторичных модификаций

4. Запись модифицированной трассы

Генерируемые модификации могут изменять существующие и добавлять новые записи, обеспечивая консистентность выходной трассы. Примеры возможных модификаций были представлены в разделе 1.1. Некоторые модификации могут снижать репрезентативность выходной трассы, так как будут приводить к изменению целевых счетчиков при дальнейшей ее обработке.

3.2 Генерация карты перемещений

На данной стадии с трассы собирается карта прав доступа для всех используемых приложением адресов. Далее производится слияние близких областей памяти с разрешенными комбинациями прав доступа. Максимальное расстояние для слияния областей является параметром командной строки. Затем производится перемещение полученных областей памяти в соответствии с новыми ограничениями адресации:

- При наличии в исходной карте прав доступа областей с запрещенными комбинациями прав, выводится ошибка - корректное преобразование трассы невозможно.
- Области, при слиянии прав доступа которых, получится комбинация прав, запрещенная новыми настройками адресации, размещаются на расстоянии превышающем новый размер страниц памяти.
- Также перемещение производится только в области памяти разрешенные новой конфигурацией.

Полученная карта перемещений используется последующими стадиями для анализа адресации и корректной замены адресов. Пример генерации карты перемещений приведен на рис. 7.

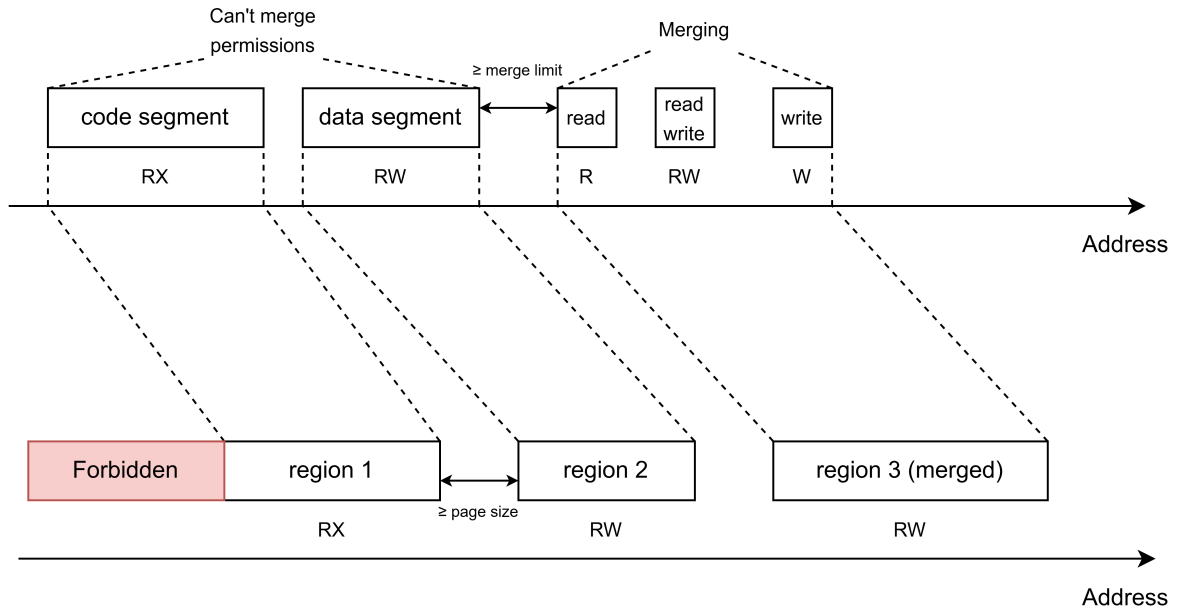


Рис. 7: Пример генерации карты перемещений.

3.3 Сбор графа потока данных

Обозначим $G = (V, E)$ - граф потока данных, где V - множество вершин графа, $E \subseteq V \times V$ - множество ребер графа. Так же обозначим: 2^E - множество всех подмножеств ребер графа. Вершины графа моделируют исполнения инструкций (иногда называемые просто инструкциями), ребра - зависимости между инструкциями по значениям. С каждым ребром ассоциируются значение и локация, по которой данное значение располагается (также называемые значением и локацией ребра). Для представления графа потока данных используется реализация списка смежности из библиотеки boost graph (boost::adjacency_list).

Boost graph позволяет ассоциировать с вершинами и ребрами пользовательские данные. Для инструкций хранятся: номер (уникальный идентификатор) инструкции, адрес инструкции, ее код. Для ребер хранятся: восьми-байтное значение и его локация (уже упомянутые ранее) - в памяти или на регистре общего назначения, модификаторы значения: тип расширения, размеры битовых сдвигов, и т. п. Заметим, что размер значения фиксированный, зависимости по значениям большего размера отображаются с помощью нескольких ребер.

На графе есть вершины, не соответствующие реальным инструкциям (вершина "начальное состояние" на рис. 8). Будем называть такие вершины псевдо-инструкциями. Такие вершины всегда являются истоками и используются для отображения различных зависимостей:

- Вершина начального состояния - для зависимостей от начального состояния памяти

или начальных значений регистров.

- Вершина внешних зависимостей по памяти - для зависимостей по памяти, модифицированной другим потоком или ОС.

Пример: Из памяти прочитано значение, не соответствующее текущему образу памяти. Данное значение могло быть записано другим потоком или ОС во время системного вызова или обработки системного исключения. На графе данная зависимость будет представлена в виде ребра, выходящего из вершины внешних зависимостей и входящего в инструкцию, прочитавшую новое значение.

- Вершина сложных зависимостей по памяти - для зависимостей по памяти от нескольких инструкций.

В данный момент полноценная поддержка таких зависимостей отсутствует.



Рис. 8: Пример графа потока данных.

Сбор графа потока данных начинается с симуляционного прохода по трассе (блок схема алгоритма представлена на рис. 9), в ходе которого собираются:

- Граф потока данных.
- Данные вершин.
- Данные ребер, кроме модификаторов значений (таких как размеры битовых сдвигов и т. п.).
- Карта, отображающая идентификаторы инструкций в дескрипторы, с помощью которых осуществляется доступ к вершинам.
- Карта зависимостей инструкций от флагов условий.

Заметим, что зависимости между инструкциями по флагам условий хранятся отдельно от графа потока данных. Данное решение позволяет разделить разнородные зависимости, что существенно упрощает обработку.

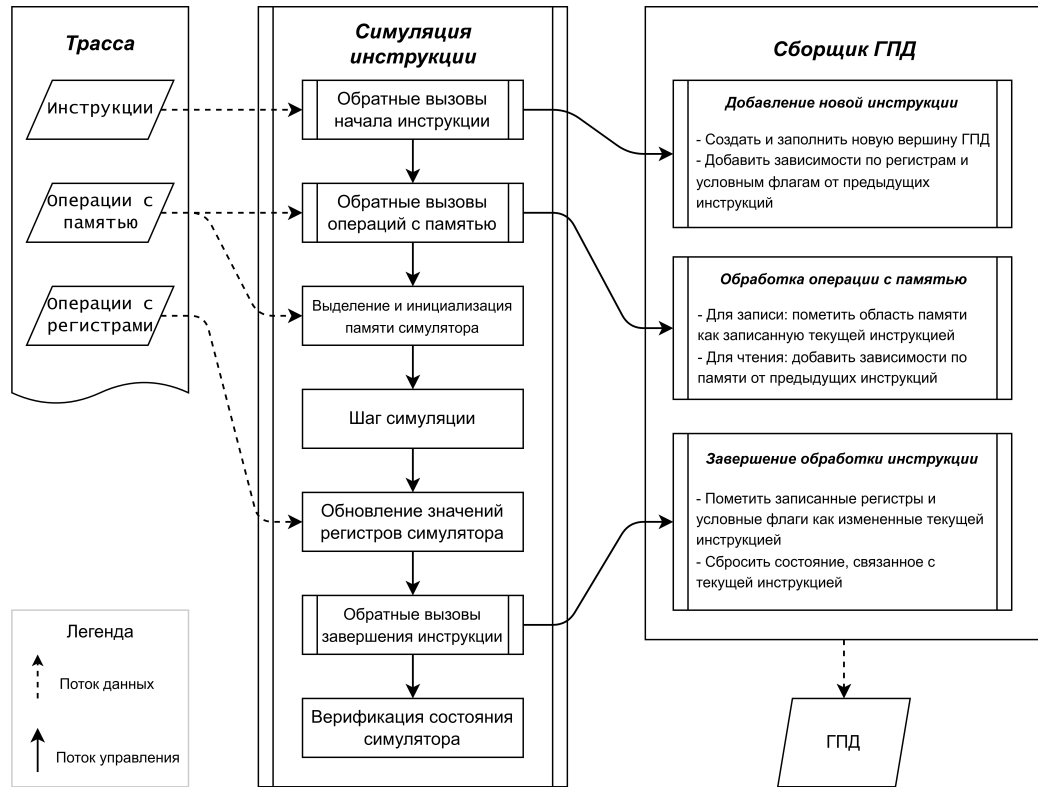


Рис. 9: Блок-схема алгоритма симуляционного прохода по трассе.

После симуляционного прохода производится ряд обходов графа, в ходе которых собираются:

- Карты подсказок для алгоритмов анализа адресов.
- Модификаторы значений для ребер графа.

Карты подсказок позволяют получать вспомогательные данные для вершин и ребер графа (примеры фрагментов ГПД и сформированных для них подсказок представлены на рис. 10):

- $\text{src_edge_hints} : E \rightarrow E$ задает порядок обратного обхода графа для алгоритма поиска адресов в ряде краевых случаев. Формируется с помощью набора эвристик.
- $\text{dst_edge_hints} : E \rightarrow 2^E = \text{src_edge_hints}^{-1}$ задает порядок прямого обхода графа для алгоритма поиска адресов в ряде краевых случаев. Задает обратное многозначное отображение для src_edge_hints .

- $\text{base_edge_hints} : V \rightarrow E$ отображает вершины в соответствующие им входные ребра базовых адресов.
- $\text{offset_edge_hints} : V \rightarrow E$ отображает вершины в соответствующие им входные ребра смещений адресов.

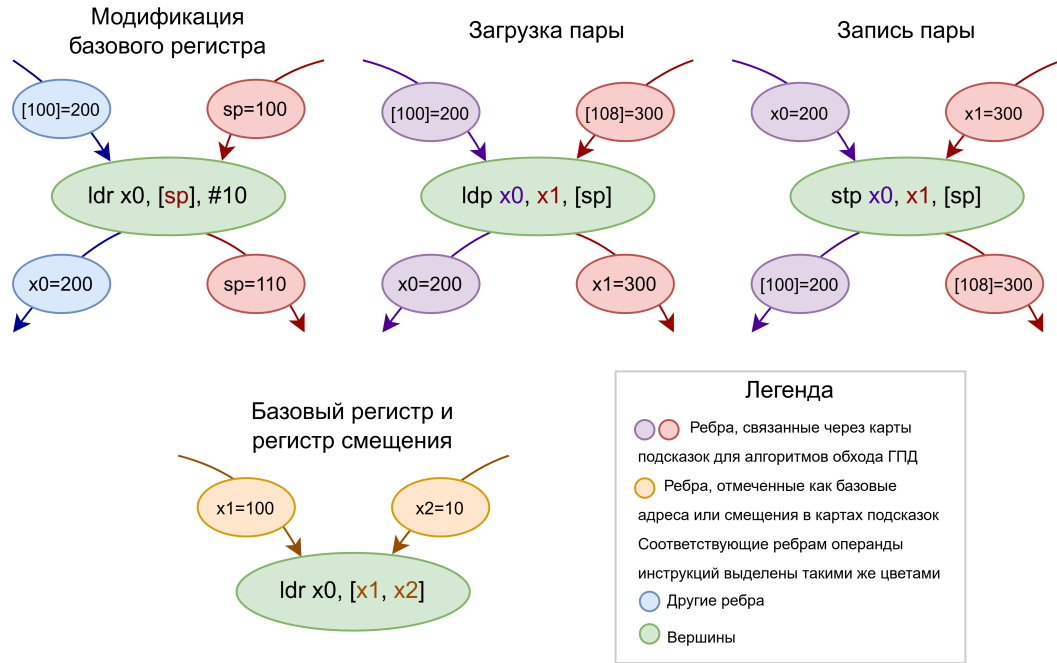


Рис. 10: Примеры фрагментов ГПД и сформированных для них подсказок.

На рис. 11 представлена UML диаграмма модуля, отвечающего за сбор и анализ графа потока данных (модуль ГПД).

- Значения регистров общего назначения

Регистровые значения, используемые в качестве адресов, должны быть заменены.

Для упрощения задачи запретим добавлять записи типа "обращение в память" и модифицировать размеры и типы обращений для существующих записей: поток данных выходной трассы должен быть похож на исходный, но с замененными адресами. Таким образом, будут использованы четыре вида модификаций для записей трассы (три из которых генерируются на рассматриваемой стадии обработки):

- Модификации начального регистрового состояния
- Модификации чтений из памяти
 - Данные могут быть заменены, если из памяти читается значение, используемое в дальнейшем в качестве адреса.
 - Адрес чтения должен быть получен из исходного с помощью карты перемещений.
 - Новые записи не добавляются.
- Модификации записей в память
 - Данные получаются из симуляции: в память может записываться перемещенный адрес (возможно, преобразованный) или старые данные.
 - Адрес записи должен быть получен из исходного с помощью карты перемещений.
 - Новые записи не добавляются.
 - Не генерируются на рассматриваемой стадии обработки: преобразования тривиальные и осуществляются при записи выходной трассы.
- Модификации записей в регистры общего назначения
 - Регистровые значения для существующих записей могут быть изменены.
 - Могут быть добавлены новые записи, исправляющие потоки управления и данных.

Генерация модификаций происходит в два этапа:

1. Поиск адресов и смещений на ГПД

На этом этапе производится поиск ребер, значения которых (возможно, после некоторых преобразований) используются в качестве адресов. Для найденных ребер генерируются первичные модификации трассы.

2. Симуляционный проход по модифицированной трассе

На этом этапе детектируются расхождения потока управления и потока данных симулятора с ожидаемым поведением на модифицированной трассе. При детектировании расхождений генерируются вторичные модификации трассы, исправляющие ошибки.

Предложенный алгоритм по построению поддерживает любые паттерны адресации в трассе, но может генерировать трассы с большим количеством компенсаций. Подробнее свойства предложенного алгоритма будут рассмотрены в дальнейшем.

3.4.1 Поиск адресов и смещений

Введем два маркера для ребер ГПД: красный маркер отмечает ребра, обработанные в ходе обратного обхода ГПД; зеленый маркер отмечает ребра, попавшие в очередь прямого обхода ГПД. В данную очередь будем помещать только ребра, еще не покрашенные зеленым маркером.

Будем называть два ребра близкими по значению если абсолютная разность их значений меньше некоторого порогового значения, являющегося гиперпараметром решения (обозначим как `ADDR_SIMILARITY_THRESHOLD`). Также будем называть ребро $e \in F \subseteq E$ ближайшим по значению к ребру g среди ребер F , если e и g близки по значению и пара (e, g) обладает минимальной абсолютной разностью значений среди всех пар $F \times \{g\}$.

Сначала с ГПД собираются ребра, значения на которых использованы в качестве адресов (адресные ребра). К таким значениям относятся: базовые адреса обращений в память; адреса переходов потока управления; адреса, используемые для операций с кешами; значения, которые сравниваются с адресами или вычитаются из них; и т.п.

Для каждого адресного ребра, строится обратный путь на ГПД, начинающийся с этого ребра. Построение выполняется последовательно. На каждом шаге для текущего конечного ребра пути (обозначим это ребро как e) выбирается следующее ребро из множества ребер, входящих в начальную вершину e . Построение пути осуществляется до выполнения одного из условий останова: достижения псевдо-инструкции; ошибки поиска следующего ребра; достижения ребра, окрашенного красным маркером; и т.п. Конечное ребро построенного пути будем называть исходным адресным ребром. Для таких ребер, а также ребер пути с локациями в памяти, генерируются первичные модификации трассы, что позволяет заменить большую часть используемых адресов (так как изменения будут распространяться по потоку данных) путем малого количества модификаций трассы. Все ребра построенного пути красятся красным маркером, а исходные адресные ребра также помещаются в очередь пря-

мого обхода ГПД (если они еще не окрашены зеленым маркером). Блок схема алгоритма построения обратных путей на ГПД представлена на рис. 12.

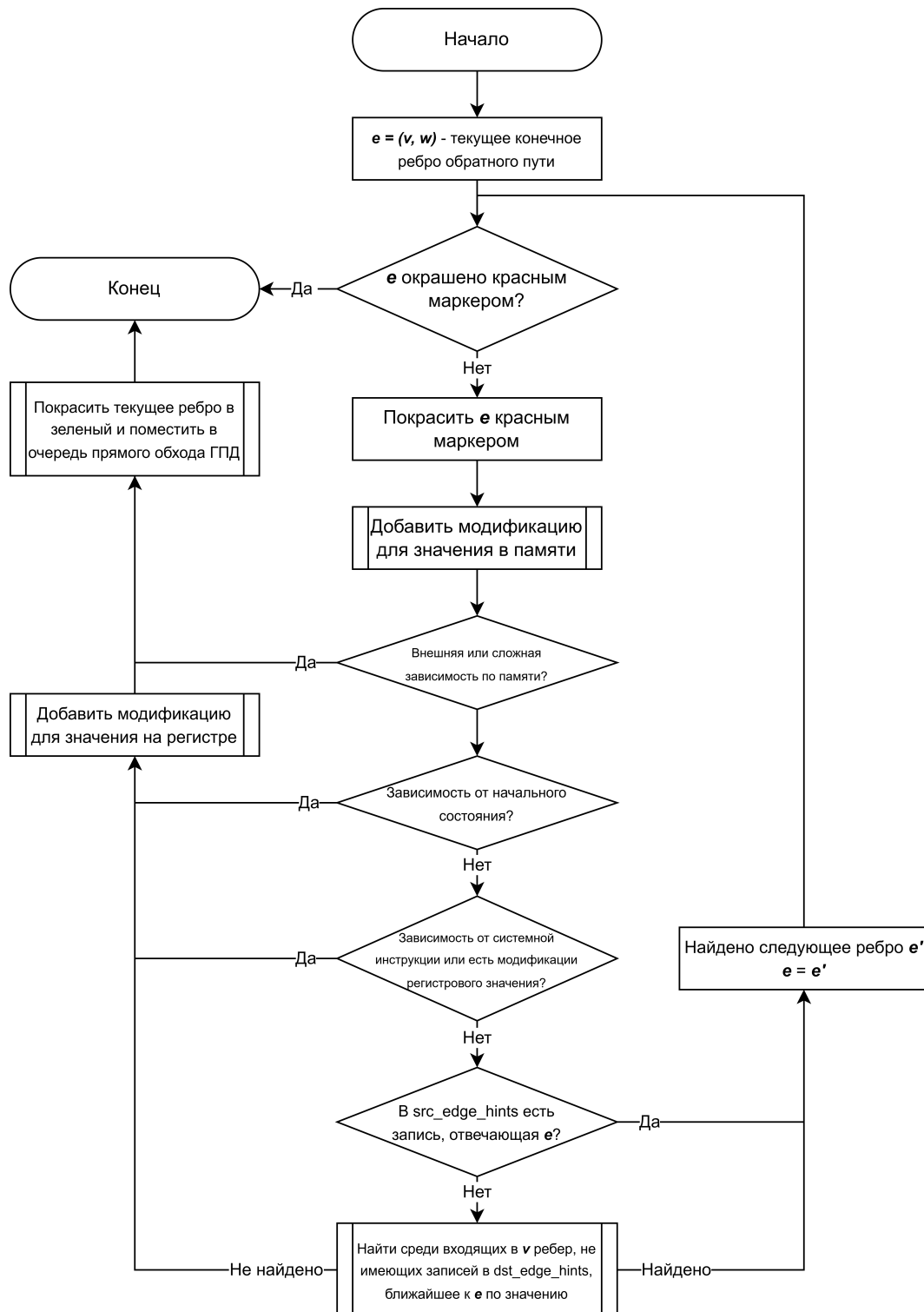


Рис. 12: Блок схема алгоритма построения обратных путей на ГПД.

Затем производится обработка ребер в очереди прямого обхода ГПД. Для каждого ребра e строится обратный путь (если он еще не был построен) и осуществляется поиск ребер для дальнейшего обхода. Найденные ребра добавляются в очередь прямого обхода:

- Ребра с такой же начальной вершиной и локацией
- Ребра, значения которых сравниваются со значением e

Если e входит в вершину v , являющуюся инструкцией сравнения, то все входные ребра v добавляются в очередь.

- Ребра, значения которых участвуют в вычитании со значением e

Сценарий аналогичен предыдущему, но значения добавляемых ребер должны быть больше некоторого порогового значения, являющегося гиперпараметром решения (обозначим как ADDR_MIN_VALUE).

- Ребра, соответствующие e в `dst_edge_hints`.
- Ребра, выходящие из v , близкие по значению к e

Данные ребра добавляются только если для e нет записей в `dst_edge_hints`.

Прямой обход ГПД позволяет найти дополнительные адресные ребра для построения обратных путей на графе и генерации первичных модификаций. Этот обход нужен, так как часть значений в трассе может не использоваться в качестве адресов напрямую, но может сравниваться с адресами, использоваться для подсчета хеш сумм адресов, и т. п. Часть таких значений будет найдена при данном обходе, что позволит в дальнейшем сделать меньше вторичных модификаций (которые всегда снижают репрезентативность трассы).

3.4.2 Симуляционный проход по модифицированной трассе

На данной стадии производится проход по трассе, аналогичный изображенному на рис. 9. Но записи трассы преобразуются в соответствии со сгенерированными модификациями. Также до и после каждого симуляционного шага потоки управления и данных симулятора проверяются на расхождения с ожидаемым поведением. При детектировании расхождения, состояние симулятора исправляется и генерируются вторичные модификации трассы, соответствующие исправлению. В случае расхождения после симуляционного шага, состояние симулятора восстанавливается и шаг делается повторно (с примененными исправлениями). Описанный подход позволяет генерировать трассы с консистентными потоками управления и данных. Блок схема алгоритма представлена на рис. 13.

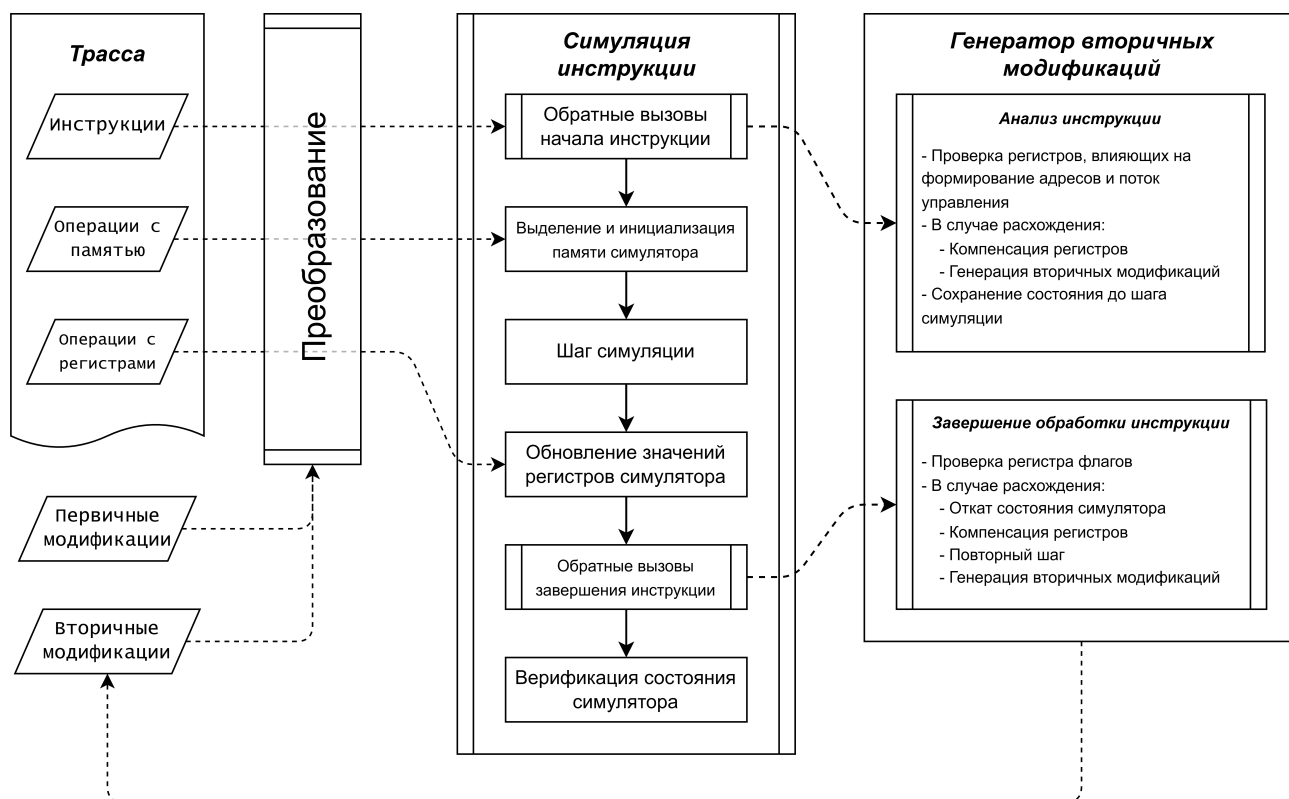


Рис. 13: Блок-схема симуляционного прохода по модифицированной трассе.

3.5 Запись модифицированной трассы

На данной стадии обработки происходит проход по исходной трассе, в ходе которого оригинальные записи преобразуются с помощью подготовленных модификаций (первичных и вторичных) и записываются в выходную трассу. Так же во время обработки происходит симуляция по модифицированной трассе, аналогичная подходу описанному в разделе 3.4.2. Симуляция используется для верификации записываемой трассы и получения данных для некоторых записей.

3.6 Симуляция по трассе

Рассмотрим подробнее симуляцию по трассе, ранее упомянутую в разделах 1.1, 3.3, 3.4.2 и 3.5. В процессе симуляции данные из трассы пробрасываются в состояние симулятора, таким образом воспроизводится записанное в трассе исполнение и собираются необходимые данные, отсутствующие в трассе (например значения регистров в определенных точках). Также обычно проверяются консистентность потока управления и данных.

Для ряда рассмотренных задач применяется симуляция по модифицированной трассе. Заметим, что модифицированная трасса при этом еще не записана в выходной файл, а конструируется налету из записей исходной трассы и подготовленных модификаций. Функ-

ционал обычной симуляции по трассе получается переиспользовать с помощью наследования и переопределения виртуальных функций. Переопределенные методы-обработчики подменяют исходные записи трассы (и/или добавляют новые), а после вызывают методы базового класса на полученных данных. Пример переопределенного метода для обработки модифицированной трассы приведен в листинге 1.

```
/// Trace-based simulation for memory record.
/// Called before related instruction simulation for data injection.
/* virtual */ void TraceSim::process_pre_mem(const trace::MemEvent &ev) {
    switch (ev.kind) {
        case trace::MemEventKind::Load:
            // Inject trace data
            m_sim->write_virt_mem(ev.addr, ev.data);
            break;
        case trace::MemEventKind::Store:
            // Allocate touched memory region
            m_sim->allocate_virt_mem(ev.addr, ev.data.size());
            break;
    }
}

/// Patched trace-based simulation for memory record.
/// Patch trace record and delegate to base method.
void PatchedTraceSim::process_pre_mem(const utl::MemEvent &ev) /* override */ {
    // Patch event
    auto new_ev = m_patches.patch_mem_event(m_current_instruction, ev);
    // Delegate
    TraceSim::process_pre_mem(new_ev);
}
```

Листинг 1: Переопределение метода-обработчика записей доступа в память

Также класс симулятора по трассе поддерживает добавление обратных вызовов (callback), что позволяет переиспользовать его для различных симуляционных задач решения. UML диаграмма симуляционного модуля представлена на рис. 14.

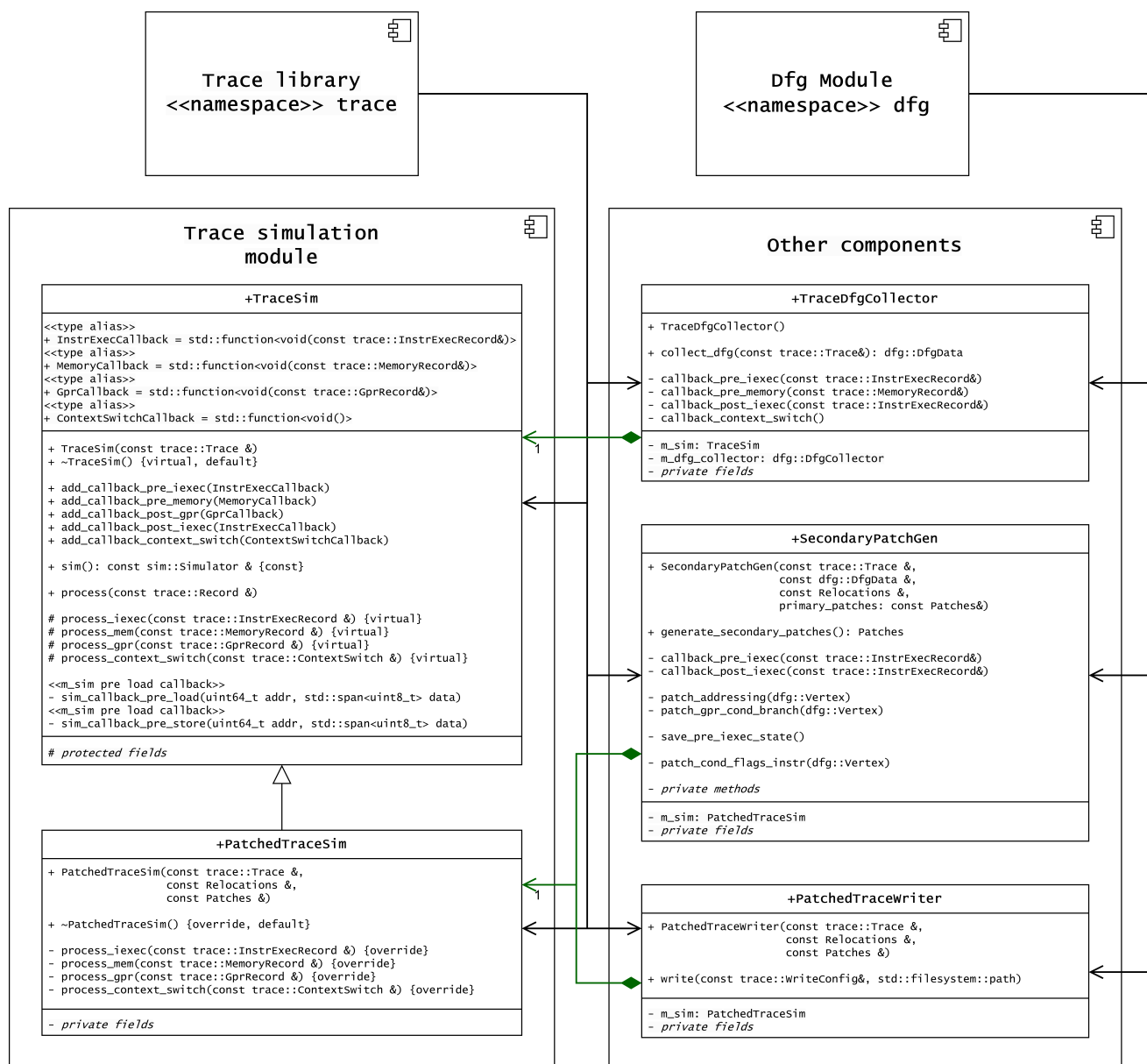


Рис. 14: UML диаграмма симуляционного модуля.

3.7 Тестирование

Для тестирования программного обеспечения (ПО) часто применяются три основных вида тестов:

- Юнит тесты (unit testing) - проверяют корректность работы отдельных функций и классов изолированно от других компонентов. Поведение внешних зависимостей имитируется с помощью объектов-подражателей (mock objects).
- Модульные тесты (module testing) - проверяют работу отдельных модулей. Тестируются корректность взаимодействий внутри модуля и его интерфейсов.
- Сквозные тесты (end to end testing) - решение тестируется полностью на целевых сценариях.

нариях и данных.

Для юнит и модульного тестирования решения используется фреймворк google test и тестовая инфраструктура используемой системы сборки CMake. Так же возможно добавление сквозных тестов с использованием реальных или синтетических трасс и инфраструктуры проекта.

Дизайн ключевых компонентов решения позволяет применять юнит и модульное тестирование, возможно после нескольких незначительных изменений. Основной задачей для поддержки подобного тестирования является отделение компонентов решения от библиотеки для чтения трасс. Это позволит осуществлять частичную обработку тестовых данных без наличия реальной трассы. Примером подобного отделения является публичный интерфейс модуля ГПД (см. рис. 11): класс *DfgCollector* принимает данные, не привязанные явно к библиотеке для чтения трасс (хотя формат данных сильно похож на формат записей трассы). Это позволяет собирать ГПД и вспомогательные данные на тестовых сценариях, а затем проверять консистентность, инварианты и выполнение тестовых условий. Для упрощения описания тестовых сценариев была реализована вспомогательная инфраструктура, в упрощенном виде представленная в листингах 2 и 3. Пример юнит теста, использующего описанную инфраструктуру представлен в листинге 4.

С помощью описанной тестовой инфраструктуры было добавлено 7 юнит тестов, покрывающих базовый функционал для работы с ГПД и ряд краевых случаев. Примененный подход показал свою эффективность: тестовые сценарии описываются легко и могут быть переиспользованы несколькими тестами для проверки различных компонентов. Полное тестирование модуля ГПД и других модулей может быть осуществлено в рамках дальнейшего развития решения.

```
/// DFG builder for testing
struct DfgBuilder {
    /// Set sim gpr
    void set_gpr(uint8_t id, uint64_t value);

    /// Simulator data getters
    uint64_t pc() const;
    uint64_t sp() const;
    uint64_t icount() const;

    /// Sink vertices getters
    dfg::Vertex init() const;
    dfg::Vertex extmod() const;
    dfg::Vertex complex() const;

    /// Info about test basic block
    struct BasicBlockInfo {
        const dfg::DfgData &dfg_data;
        /// Global icount range
        uint64_t start_ic = 0;
        uint64_t end_ic = 0;

        // Helper getters ...
    };

    /// Info about load
    struct Ld {
        uint64_t addr = 0;
        std::vector<uint8_t> data;
    };

    /// Info about store
    struct St {
        uint64_t addr = 0;
        std::vector<uint8_t> data;
    };

    /// Add specified basic block & return aggregated info about it
    template <class... Args>
    BasicBlockInfo bb(Args &&...args);

    /// Finalize DFG building
    void finalize();
};
```

Листинг 2: Класс для генерации тестовых ГПД.

```

/// Fixture for `DfgCollector` tests
struct DfgCollectTest : public DfgBuilder, public ::testing::Test {
protected:
    /// Check that vertex with specified icount exists and return descriptor for it
    dfg::Vertex expect_vertex(uint64_t ic);
    /// Check that specified edge exists and return descriptor for it
    dfg::Edge expect_edge(dfg::Vertex src, dfg::Vertex dst,
        uint64_t value, uint8_t gpr_id, uint8_t op_id);
    /// Check that specified edge exists and return descriptor for it
    dfg::Edge expect_edge(dfg::Vertex src, dfg::Vertex dst,
        uint64_t value, const MemoryLocation &loc);
    /// Check that edge `src` has a destination hint pointing to `dst` edge
    void expect_src_dst_hint(dfg::Edge src, dfg::Edge dst);
};

```

Листинг 3: Класс-фикстура для юнит тестирования класса DfgCollector.

```

/// Test for `ldp` instruction processing based on `DfgCollectTest` fixture
TEST_F(DfgCollectTest, ldpImmediateOffset) {
    auto bb1 = bb(
        "ldp x3, x4, [sp, #16]",
        Ld{sp() + 16, {uint64_t{10}, uint64_t{20}}},
        "add x5, x3, x4"
    );
    DfgBuilder::finalize();

    expect_edge(init(), bb1.vertex(0), sp(), uint8_t{31}, uint8_t{2});
    // Check edges from initial memory to `ldp`
    auto edge_init_to_ldp_mem0 = expect_edge(init(), bb1.vertex(0), 10, ....);
    auto edge_init_to_ldp_mem1 = expect_edge(init(), bb1.vertex(0), 20, ....);
    // Check edges from `ldp` to `add`
    auto edge_ldp_to_add_x3 = expect_edge(bb1.vertex(0), bb1.vertex(1), 10, ....);
    auto edge_ldp_to_add_x4 = expect_edge(bb1.vertex(0), bb1.vertex(1), 20, ....);

    // Check hints for related edges
    expect_src_dst_hint(edge_init_to_ldp_mem0, edge_ldp_to_add_x3);
    expect_src_dst_hint(edge_init_to_ldp_mem1, edge_ldp_to_add_x4);
}

```

Листинг 4: Пример юнит теста для класса DfgCollector.

3.8 Оценка решения

В данном разделе будет произведена оценка решения.

3.8.1 Пространственная и временная сложность

Рассмотрим алгоритмическую сложность решения. Размеры трассы и количество итераций при проходе по ней пропорциональны количеству инструкций в трассе $N = |V|$.

Рассмотри временную и пространственную сложность стадий обработки трассы:

- Генерация карты перемещений

Временная сложность - $O(N)$ (полный проход по трассе). Пространственная сложность - $O(N)$ (агрегация данных с записей трассы).

- Сбор ГПД

Временная сложность - $O(N)$ (полный проход по трассе). Пространственная сложность - $O(N + K)$. Где $K = |E|$ - количество ребер ГПД.

- Генерация модификаций трассы

На данной стадии производится два обхода ГПД: построение обратных путей и прямой обход. При этом ни одно ребро не обходится дважды, так как обработанные ребра отмечаются маркерами. Временная сложность обходов графа - $O(N + K)$. Также в конце производится симуляционный проход по модифицированной трассе, временная сложность - $O(N)$.

Результирующая временная сложность - $O(N + K)$

На данной стадии генерируются модификации трассы, количество которых пропорционально количеству инструкций и количеству ребер ГПД. Пространственная сложность - $O(N + K)$.

- Модификация трассы

Временная сложность - $O(N)$ (полный проход по трассе). Пространственная сложность - $O(1)$.

Заметим, что выполняется асимптотическая оценка: $K = O(N)$. Таким образом, пространственная и временная сложность обработки трассы $O(N + K) = O(N)$.

3.8.2 Оценка репрезентативности трасс

Для оценки репрезентативности выходных трасс будет использоваться статистика произведенных модификаций. Рассмотрим различные модификации трассы и их влияние на репрезентативность:

- Замена адреса для операции с памятью - не влияет на репрезентативность, так как не требует компенсации при дальнейшей обработке выходной трассы.
- Замена значения для записи в память - не требует компенсации (поток данных консистентен).
- Замена значения для чтения из памяти - может влиять на репрезентативность в некоторых сценариях. При чтении значения, не соответствующего текущему образу памяти рассматриваемого потока, может потребоваться компенсация, аналогичная компенсации для внешней модификации памяти. Заметим, что если в оригинальной трассе уже присутствовала внешняя модификация памяти, модификация не повлияет на репрезентативность.
- Замена значения регистра в начальном состоянии - не требует дополнительной компенсации.
- Замена значения для записи в регистр общего назначения - не требует дополнительной компенсации.
- Добавление записи в регистр общего назначения - требует компенсации, будет снижать репрезентативность.

Таким образом, для оценки влияния на репрезентативность трасс будем использовать кумулятивную метрику N_{comp} - количество модификаций, приводящих к появлению дополнительных компенсаций при обработке трассы. Обозначим также $P_{comp} = \frac{N_{comp}}{|V|}$ - количество компенсируемых модификаций на инструкцию трассы.

3.8.3 Профилировка решения по памяти

Для получения грубой оценки потребления памяти различными компонентами использовалась утилита `heaptrack`, позволяющая отследить объемы выделений памяти различными участками кода. Данные `heaptrack` для набора тестовых трасс показывают, что в коде библиотеки `Boost Graph` выделяется больше 90 процентов используемой памяти.

3.8.4 Тестовые запуски на целевых трассах

В данном разделе будут представлены результаты тестовых запусков на целевых трассах заказчика (таблица 1). Рассматриваются следующие метрики:

- N - количество инструкций в трассе.
- N_{comp} - количество компенсируемых модификаций.
- $P_{comp} = \frac{N_{comp}}{N}$ - количество компенсируемых модификаций на инструкцию трассы.
- RSS_{max} - максимальный объем резидентной памяти (resident set size), занятый при модификации трассы.
- $P_{RSS} = \frac{RSS_{max}}{N}$ - объем резидентной памяти на инструкцию трассы.
- T - время обработки трассы.
- $V = \frac{N}{T}$ - средняя скорость обработки (инструкций в секунду).

Таблица 1: Результаты тестовых запусков на целевых трассах заказчика

Трасса (тип нагрузки)	N	N_{comp}	P_{comp}	RSS_{max} , Гб	P_{RSS} , $\frac{Гб}{10^6}$	T , с	$V, \frac{1}{с}$
LogicTrace1 (вычислительная)	$1.7 \cdot 10^7$	$3.7 \cdot 10^5$	$2.1 \cdot 10^{-2}$	7.65	0.44	162.5	$1.07 \cdot 10^5$
LogicTrace2 (вычислительная)	$1.6 \cdot 10^7$	$4.0 \cdot 10^5$	$2.5 \cdot 10^{-2}$	6.91	0.44	149.3	$1.06 \cdot 10^5$
LogicTrace3 (вычислительная)	$4.1 \cdot 10^7$	$9.5 \cdot 10^4$	$2.3 \cdot 10^{-3}$	16.71	0.41	408.7	$1.00 \cdot 10^5$
LogicTrace4 (вычислительная)	$1.6 \cdot 10^7$	$7.4 \cdot 10^3$	$4.5 \cdot 10^{-4}$	6.91	0.43	160.0	$1.05 \cdot 10^5$
LogicTrace5 (вычислительная)	$3.4 \cdot 10^7$	$1.3 \cdot 10^3$	$4.1 \cdot 10^{-5}$	13.38	0.40	321.3	$1.04 \cdot 10^5$
GfxTrace1 (графика)	$1.0 \cdot 10^7$	$7.0 \cdot 10^3$	$6.7 \cdot 10^{-4}$	4.47	0.43	99.3	$1.05 \cdot 10^5$

Обозначим: M - количество тестовых трасс. Рассчитаем средние значения для P_{comp} , P_{RSS} и V .

Для P_{comp} приведено среднее арифметическое - доля компенсированных инструкций характеризует качество выходной трассы, подобное усреднение позволит оценить долю компенсаций в случайно выбранной трассе. При этом не учитываются размеры тестовых трасс, так как на долю компенсаций в первую очередь влияют особенности потоков данных и управления. Также для P_{comp} приведен 50-й перцентиль (медиана), так как на ряде тестовых трасс доля компенсаций существенно выше, что может говорить о наличии большого количества еще не поддержанных краевых случаев.

$$\overline{P_{comp}} = \frac{1}{M} \sum_{i=1}^M P_{comp_i} = 8.3 \cdot 10^{-3}$$

$$P_{50}(P_{comp}) = 6.7 \cdot 10^{-4}$$

P_{RSS} также усреднено с помощью среднего арифметического - потребление памяти линейно от количества инструкций (остальные накладные расходы пренебрежимо малы). Значит потребление памяти на одну инструкцию также будет в первую очередь зависеть от свойств потоков управления и данных (например от количества ребер на одну инструкцию), но не от размеров трасс. Подобная оценка позволит оценить максимальное суммарное количество инструкций в наборе трасс, который можно обрабатывать одновременно в условиях ограниченной памяти.

$$\overline{P_{RSS}} = \frac{1}{M} \sum_{i=1}^M P_{RSS_i} = 0,42 \frac{\text{Гб}}{10^6}$$

Для V приведено взвешенное гармоническое среднее, являющееся средней скоростью обработки всего массива тестовых трасс.

$$\overline{V} = \frac{\sum_{i=1}^M N_i}{\sum_{i=1}^M T_i} = 103334 \frac{1}{\text{с}}$$

3.9 Возможные направления дальнейшего развития решения

3.9.1 Поддержка краевых случаев

Компенсация ряда часто встречающихся краевых случаев может быть оптимизирована с помощью дополнительного анализа или добавления специализированных записей, компенсация которых может быть упрощена. Примеры возможных оптимизаций:

- 32-битные адресные значения

Ряд 64-битных архитектур поддерживает использование 32-битных адресов для оптимизации или поддержки обратной совместимости. При модификации такого адреса, новое значение может не помещаться в 32 бита. В подобных случаях появляются типовые компенсации: выставление нескольких старших бит адреса (часто одинаковых от обращения к обращению). Для таких случаев можно добавлять в трассу специализированные записи, которые позволят компенсировать только эти старшие биты, что может существенно улучшить репрезентативность для ряда сценариев. Подобная оптимизация требует поддержки в обработчиках трасс.

- Хеширование адресов

Для оптимизации компенсаций хешей адресов можно детектировать хеш функции по символьной информации (в случае полноценного вызова) и/или с помощью специализированного алгоритма, детектирующего хеширование на ГПД (при отсутствии символьной информации или инлайнинге) - с помощью методов сопоставления сигнатур графов, символьного исполнения и SAT решателей.

3.9.2 Оптимизация по памяти

В разделе 3.8.4 приведено потребление RAM памяти во время тестовых запусков на реальных трассах исполнения заказчика (таблица 1, также приведено усредненное значение). Подобное потребление не позволяет обрабатывать трассы с количеством инструкций порядка $1 \cdot 10^9$ и более на типовых серверах, используемых в индустрии. Также высокое потребление памяти существенно ограничивает возможности для параллельной обработки наборов трасс. Также в разделе 3.8.4 было показано, что основную долю потребляемой RAM занимает ГПД.

Для снижения потребления RAM памяти можно использовать специализированные библиотеки для работы с большими графами. Такие решения хранят графы на диске и подкачивают данные в RAM при необходимости. Также могут быть применены менее специализированные библиотеки: библиотека STXXL, предоставляющая реализации для типовых C++ контейнеров, оптимизированных под большие объемы данных; библиотеки LLNL Metall и LLNL Umap, предоставляющие высокоуровневые интерфейсы для выделения памяти через отображаемые файлы (ОС может выгружать такую память на диск) и контроля над выгрузкой страниц соответственно, что позволяет использовать эти библиотеки для ограничения используемой RAM.

Другим вариантом решения проблемы является отказ от хранения ГПД для всей трассы. Возможна блочная обработка трассы, со сбором ГПД только для обрабатываемого в данный момент блока. Такой подход потребует многократного сбора одних и тех же участков графа для осуществления всех необходимых обходов графа и обходов трассы, использующих данные графа. Подобное многократное восстановление данных может существенно замедлить обработку трасс, в таком случае потребуются дополнительные оптимизации.

3.9.3 Поддержка векторных регистров

В данный момент на ГПД добавляются только зависимости по памяти и регистрам общего назначения. При этом векторные регистры также могут использоваться для формирования адресов. Добавление информации о значениях векторных регистров на ГПД позволит улучшить качество генерируемых модификаций (можно будет отследить адресные значения на векторных регистрах) и поддержать инструкции, напрямую использующие эти регистры для формирования адресов (например scatter store и gather load инструкции из расширения SVE системы команд ARM64).

3.9.4 Модификация криптографических подписей адресов

Представленное решение может быть использовано не только для изменения образа памяти трасс, но и для других задач, требующих поиска адресов в трассе. Одной из подобных задач является замена криптографических подписей адресов.

Криптографические расширения систем команд, такие как PAuth (ARM64), позволяют подписывать используемые адреса и верифицировать их подпись непосредственно перед использованием. Это позволяет защитить систему от ряда кибер атак, основанных на подмене адресов потока управления.

При анализе трасс, содержащих PAuth инструкции, появляется необходимость переподписать адреса из трассы, так как ключ шифрования трассируемой системы может быть неизвестен. Локации подписанных адресов можно искать с помощью описанных в данной работе алгоритмов, после их незначительной корректировки.

4 Заключение

В рамках работы было представлено решение для модификации трасс исполнения с целью изменения записанных образов памяти трассируемых сценариев. Реализованы многопроходные алгоритмы для сбора и анализа графа потока данных, которые позволяют находить локации адресов в трассе и заменять их в соответствии с ограничениями новых настроек адресации, используемых при дальнейшем анализе трасс. Решение позволило успешно модифицировать целевые трассы заказчика, которые затем прошли верификацию на функциональном симуляторе и были использованы для анализа производительности разрабатываемых систем с настройками адресации, отличающимися от изначальных настроек адресации (настроек трассируемой платформы).

Была разработана инфраструктура для описания тестовых сценариев для юнит- и модульных тестов. На основе данной инфраструктуры были написаны базовые тесты для модуля сбора ГПД. Возможно применение данной инфраструктуры для покрытия тестами других модулей.

Рассмотрена методология для оценки репрезентативности (качества) выходных трасс. Представлены метрики, собранные в результате тестовых запусков решения на целевых трассах исполнения заказчика: скорость обработки (инструкций в секунду); потребление памяти (максимальный объем резидентной памяти, расход на одну инструкцию); метрики для оценки репрезентативности.

Приведены возможные направления для дальнейшего развития решения.

Список литературы

1. Atlantis: Improving the Analysis and Visualization of Large Assembly Execution Traces / Н. N. Huang [и др.] // 2017 IEEE International Conference on Software Maintenance and Evolution (ICSME). — 2017. — С. 623—627. — DOI: 10.1109/ICSME.2017.23.
2. *Bendersky E.* Position Independent Code (PIC) in shared libraries. — 11.2011. — URL: <https://eli.thegreenplace.net/2011/11/03/position-independent-code-pic-in-shared-libraries/> (дата обр. 16.04.2026). eli.thegreenplace.net.
3. Checkpoint Extraction Using Execution Traces for Intra-task DVFS in Embedded Systems / Т. Tatematsu [и др.] // 2011 Sixth IEEE International Symposium on Electronic Design, Test and Application. — 2011. — С. 19—24. — DOI: 10.1109/DELTA.2011.13.
4. *Pappas V., Polychronakis M., Keromytis A. D.* Dynamic Reconstruction of Relocation Information for Stripped Binaries // Research in Attacks, Intrusions and Defenses / под ред. А. Stavrou, Н. Bos, G. Portokalidis. — Springer International Publishing, 2014. — С. 68—87. — ISBN 978-3-319-11379-1.
5. PinPlay: A framework for deterministic replay and reproducible analysis of parallel programs / Н. Patil [и др.] //. — 04.2010. — С. 2—11. — DOI: 10.1145/1772954.1772958.
6. *Prähofer H., Schatz R., Grimmer A.* Behavioral model synthesis of PLC programs from execution traces // Proceedings of the 2014 IEEE Emerging Technology and Factory Automation (ETFA). — 2014. — С. 1—5. — DOI: 10.1109/ETFA.2014.7005259.
7. *С.А. Л.* Исследование и оптимизация применения трасс исполнения приложения для статической бинарной трансляции под RISC архитектуры : дис. . . . канд. / С.А. Лисицын. — ФГАОУ ВО Московский физико-технический институт (национальный исследовательский университет), 2022.